ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Backend-сервис прогнозирования продаж

Nonton • Склады × артикулы • Горизонт 64 дня

НАЗНАЧЕНИЕ Документ можно передать backend-команде как основу задач на реализацию FastAPI + Docker. Граница ответственности ML и backend, контракты интеграции и критерии приёмки сформулированы ниже.

Параметр	Значение
Версия документа	1.0 · 15 октября 2026
Статус	Проект технического задания; решения из ТЗ и командные допущения отмечены отдельно
Заказчик	Команда хакатона «Прогноз продаж по складам и товарам»
Исполнители	Backend / MLOps / ML Engineering
Основная цель	Автономный сервис, принимающий sales + stock CSV и выдающий 64-дневный прогноз

Что должно получиться

- Готовая модель обучается заранее. Backend НЕ обучает модель внутри API-запроса.
- POST /predict принимает два CSV, рассчитывает 562 496 прогнозных строк и возвращает CSV.
- GET /predict и GET /predict/all отдают сохранённый последний успешный прогноз.
- Автономный Docker-образ, без сети на инференсе, с тестами и README.

Основание: официальное ТЗ Nonton на 27 страниц, разделы 1–10, включая бонусное задание на 8 баллов; также зафиксированная командой архитектура Horizon-Aware Global Direct LightGBM.

1. Краткое резюме и статусы требований

АРХИТЕКТУРА Horizon-Aware Global Direct LightGBM: единая предобученная модель получает исторические признаки пары склад–товар, номер горизонта h=1...64 и известные календарные признаки. Предыдущие прогнозы не используются как последующие лаги.

В этом документе используются три статуса:

- «ТЗ» — прямое требование организаторов хакатона.
- «Решение» — выбранная инженерная реализация команды; её можно изменять при согласовании.
- «Уточнить» — противоречие или неполнота исходного ТЗ; до ответа реализуем согласованное поведение по умолчанию.

Показатель	Значение / правило	Статус
Горизонт	64 календарных дня от последней даты sales	ТЗ
Разрез	date + warehouse_name + article	ТЗ
Каталог	11 складов × 799 артикулов, полный декартов грид	ТЗ / уточнить
Количество строк	11 × 799 × 64 = 562 496	ТЗ
Прогноз	Числовой, без NaN/Inf и отрицательных значений	ТЗ + усиление
Минимальная история	≥ 180 календарных дней, более длинная допускается	ТЗ
Модель	Предобученная LightGBM, без online-fit	Решение
Кэш	Последний успешно рассчитанный прогноз в RAM	Решение
Форматы	POST: CSV; GET all: CSV и JSON	ТЗ

2. Границы ответственности

ML-команда передаёт	Backend-команда реализует
Модель / веса в согласованном формате	Загрузку модели на старте контейнера и готовность сервиса
Единый модуль build_features / predict_64	HTTP-обёртку, безопасный приём файлов, маршруты, кэш
Артефакты: каталоги ID, конфигурация, список признаков	Проверку совместимости файлов и модельных артефактов
Правила fallback / postprocessing	Вызов общего ML-инференса без дублирования алгоритма
Валидационные примеры и эталонный прогноз	Интеграционные и контрактные тесты API, нагрузочные проверки
Метрики и режим валидатора ML	Контейнер, версии зависимостей, README и инструкции сборки

НЕ ВХОДИТ В BACKEND Подбор объективной функции, обучение LightGBM, отбор признаков по качеству, кросс-валидация и оптимизация leaderboard-score. Это зона ML-команды. Backend гарантирует корректный и воспроизводимый вызов предобученного inference.

3. Входные данные и схема CSV

3.1. sales — обязательные колонки

Колонка	Тип / смысл
date	YYYY-MM-DD; дата продажи
composite_division	Строка; департамент (только исторический признак, при прогнозе агрегируется)
cities_name	Строка; город
warehouse_name	Строка; склад
category_lvl_1 / _2 / _3	Строки; уровни категории товара
article	Строка; артикул
characteristic	Строка; характеристика товара
quantity	Число ≥ 0; продажи (агрегируется суммой)

sales_sum	Число; историческая сумма продаж на анонимизированной шкале
cost_price	Число; историческая цена на анонимизированной шкале
discount_sum	Число; историческая скидка
volume_weight	Число; объёмный вес

3.2. stock — обязательные колонки

Колонка	Тип / смысл
date	YYYY-MM-DD; дата измерения остатка
warehouse_name	Строка; склад
char_article	Строка; соответствует sales.article; переименовать при подготовке
stock	Число ≥ 0; остаток на соответствующий день

3.3. Нормализация и качество данных

1. CSV читаются строго с сохранением текстовых ID (запрещено терять ведущие нули артикулов). Пустые файлы и сломанный синтаксис CSV — HTTP 400.
2. Проверяются обязательные колонки, невалидные даты, пустые ключевые ID, отрицательные или нечисловые quantity/stock, отсутствие истории ≥180 дней — HTTP 422.
3. Для sales одинаковые (date, warehouse_name, article), относящиеся к разным департаментам, агрегируются по quantity=sum; прочие признаки агрегируются по согласованным правилам ML.
4. Для stock конфликтующие дубли на одном (date, warehouse_name, char_article) не суммируются автоматически. Выдать 422 до согласования семантики дубликатов.
5. Отсутствие строки sales можно интерпретировать как quantity=0 только при подтверждённом календарном покрытии выгрузки. Отсутствующий stock означает «нет измерения», а НЕ 0.
6. Дата старта прогноза — ровно max(sales.date) + 1 день. Остатки после этой даты не должны попадать в исторические признаки.

> **УТОЧНИТЬ** В ТЗ минимальная глубина «180 дней» проверяется по диапазону дат. Рабочая реализация: max(date) − min(date) + 1 ≥ 180 календарных дней; дополнительно фиксируем разрывы покрытия и передаём их в логи. Проверка 180 уникальных дат как обязательная без подтверждения может ошибочно отклонять событийные данные.

4. Интеграционный контракт ML-инференса

Backend должен вызывать единую библиотечную функцию, которую также используют ML-скрипты для основного submission. Нельзя переписывать feature engineering отдельно в app/.

```
predict_64(
    sales: pandas.DataFrame,
    stock: pandas.DataFrame,
    bundle: ModelBundle,
) -> pandas.DataFrame


# Возвращаемые колонки строго:
# date, warehouse_name, article, quantity
```

Этап	Обязательное поведение
Startup	Загрузить веса, список признаков, каталог warehouses/articles, словари категорий и config; проверить модель/schema version.
Snapshot	Для каждой пары сформировать исторические признаки только по данным на cutoff; основное окно — последние 180 календарных дней.
Grid	Создать 64 календарные даты × 11 складов × 799 артикулов; должны учитываться и исторически неактивные пары.
Forecast	Передать h=1…64 и календарные признаки будущих дат в одну предобученную модель; без подстановки собственных прогнозов в лаги.

Stock	Использовать только известные исторические остатки; нулевой остаток на cutoff НЕ обнуляет автоматически все будущие 64 дня.
Postprocess	Проверить конечность и числовой тип; применить clip ≥0; необоснованное округление не использовать.
Final validation	Ровно 562 496 строк, 64 подряд идущих даты, уникальные ключи, отсутствие NaN/Inf/отрицательных значений.

4.1. Контракт модели и конфигурации

```
model/
  lightgbm.txt          # или model.joblib, один закреплённый формат
  config.json           # model_version, schema_version,
                        # features[], history_days, horizon=64
  catalogs.json         # warehouses[11], articles[799], categories
  checksums.json        # контрольные суммы модельных файлов (опционально)
```

Все обученные статистики/кодировщики, необходимые для inference, должны входить в model/ или в зафиксированный ML-модуль; онлайн-доступа к обучающей выборке нет. Сохранённые справочники сами по себе допустимы, если получены из разрешённого набора данных.

ВАЖНО Основной submission и контейнер можно сравнивать на идентичность только при одинаковой истории входных данных, версиях модели, наборе признаков и постобработке. «Не хуже» в официальных баллах нельзя гарантировать одними инженерными мерами — нужен backtest по одинаковым CSV.

5. HTTP API: обязательные эндпоинты

5.1. POST /predict — загрузка истории и расчёт

Атрибут	Контракт
Метод / путь	POST /predict
Content-Type	multipart/form-data
Поля формы	sales: File (CSV), stock: File (CSV); оба обязательны
200 OK	text/csv; файл с date,warehouse_name,article,quantity, 562 496 строк
400 Bad Request	Пустой или синтаксически некорректный CSV
422 Unprocessable Entity	Схема/типы/даты/история/отрицательные числа/недопустимые ключи
409 Conflict	Новый POST уже выполняется (решение для защиты от конкуренции)
Время	≤ 60 минут на полный набор — лимит Т3; внутренний ориентир установить после бенчмарка

```
curl -X POST http://localhost:8000/predict \
  -F "sales=@sales_last_180.csv" \
  -F "stock=@stock_last_180.csv" \
  -o forecast.csv
```

5.2. GET /predict — фильтрация

Query-параметры: article (необязательный), warehouse (необязательный). Хотя бы один обязателен; иначе 400. При обоих параметрах вернуть только одну пару на 64 дня; при одном — все подходящие пары. Выход: text/csv с теми же четырьмя колонками.

```
GET /predict?article=char_article_1
GET /predict?warehouse=warehouse_name_3
GET /predict?article=char_article_1&warehouse=warehouse_name_3
```

Решение: GET до первого успешного POST возвращает 409; запрос несуществующего ID — 404. Эти статусы не регламентированы ТЗ напрямую и должны быть согласованы внутри команды.

5.3. GET /predict/all — весь сохранённый прогноз

Запрос	Ответ
GET /predict/all?format=csv	StreamingResponse, text/csv, 562 496 записей
GET /predict/all?format=json	JSON-массив объектов; предпочтительно потоковая выдача, без дублирования полного массива в памяти
format не csv/json	400 Bad Request
GET до POST	409 Conflict — кэш ещё пуст

5.4. GET /health — проверка работоспособности

```
GET /health
200 application/json
{"status":"ok","model_loaded":true,"model_version":"..."}
```

GET /health не запускает расчёт прогноза. При невозможности загрузить модель контейнер не должен успешно стартовать в состоянии «готов».

5.5. Формат ошибок

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{"detail":"Недостаточно истории: 179 дней, минимум 180",
 "code":"VALIDATION_ERROR"}
```

Для 400 и 422 требуем единый JSON-объект с текстовым detail и code. Нужно переопределить также встроенные ошибки валидации FastAPI (например, отсутствующий multipart-file), иначе они могут вернуть другой формат.

6. Семантика ответа, кэш и конкурентный доступ

7. Прогноз вычисляется только при POST. После успеха POST кэш заменяется атомарно. При ошибке POST предыдущий успешный прогноз остаётся доступен.
8. GET обслуживают последний завершённый прогноз. В MVP используем один Uvicorn worker и lock вокруг обновления кэша; это предотвращает разные кэши у воркеров.
9. Параллельные тяжёлые POST не запускаются: либо сериализуются, либо новый получает 409. Выбранный вариант прописать в README; предпочтение — 409 без очереди для хакатона.
10. CSV отдаётся чанками (например, 10–20 тыс. строк) через StreamingResponse. JSON — потоковым массивом объектов, без полного to_dict() на 562 496 строк.
11. Контрольный размер CSV: 562 496 строк данных + 1 строка заголовка = 562 497 строк файла.
12. После рестарта кэш в RAM пуст; требуется новый POST. Это ограничение необходимо прямо указать в README.
13. Нельзя менять прогноз между повторными одинаковыми запросами при фиксированной модели и входных CSV. Файлы модели и конфигурация неизменны в ходе работы процесса.

6.1. Политика неизвестных товаров и складов

> **РЕШЕНИЕ V1** Используется фиксированный каталог из model/catalogs.json: ровно 11 складов и 799 артикулов. Входной неизвестный warehouse/article — 422 с перечнем первых неизвестных ID; ранее отсутствовавшая в sales, но допустимая по каталогу пара получает прогноз через ML/fallback. Динамический cold start не входит в MVP.

ТЗ не определяет, как добавлять новые ID, одновременно требуя строго 562 496 строк. Поэтому динамическое расширение каталога следует проектировать отдельной версией API, не смешивая с хакатонным контрактом.

7. Docker, лимиты, автономность и эксплуатация

Ограничение	Что необходимо обеспечить
Сборка	≤ 60 минут; зависимости устанавливаются при build, не при первом запросе
Размер образа	≤ 30 GB; не копировать тренировочный датасет, ноутбуки, кэш и промежуточные файлы
ОЗУ	≤ 64 GB; целевой процесс ограничить и замерить peak RSS на полном POST
GPU	Не обязателен; базовый сервис должен работать CPU-only
POST	≤ 60 минут на полный инференс
GET	≤ 10 минут на запрос; стремиться существенно быстрее
Сеть во время infer	Запрещена; веса и словари лежат в образе
Детерминированность	Зафиксированы версии библиотек, model/config и порядок ключей/признаков
Docker workers	1 worker для in-memory кэша; многопроцессность требует внешнего общего хранилища

7.1. Рекомендуемый образ

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY app/ ./app/
COPY model/ ./model/
EXPOSE 8000
CMD ["uvicorn","app.main:app","--host","0.0.0.0","--port","8000","--workers","1"]
```

Убрать из build context исходные train CSV, .venv, .git, ноутбуки, __pycache__, тестовые большие файлы через .dockerignore. Не загружать веса из интернета при старте контейнера.

8. Тестирование и критерии приёмки

8.1. Контрактные и негативные тесты

Тест	Ожидаемый результат
Валидные sales/stock ≥ 180 дней	200; CSV с 562 496 строками данных, 64 последовательными датами
Повторный POST с теми же байтами	Детерминированное совпадение ключей и значений прогноза в числовом допуске
GET фильтр article	64 × число складов, соответствующих каталогу
GET фильтр warehouse	64 × 799 строк
GET фильтр article + warehouse	64 строки
GET без параметров	400 и единый формат JSON-ошибки
GET /predict/all csv/json	Корректный полный прогноз в обоих форматах
Пустой CSV / сломанный CSV	400
Нет обязательной колонки	422
Невалидный YYYY-MM-DD	422
История 179 календарных дней	422
Отрицательные quantity/stock	422
Нечисловые quantity/stock и NaN	422
Конфликтующие дубли stock	422 до уточнения политики
Нет NaN/Inf/негативных результатов	Проверка генератора после predict

Новые ID вне каталога	422 (строгий V1)
GET до первого POST	409 (решение V1)
Нет сети при POST	Сервис работает в Docker --network none при доступе через docker exec / локальный тест

8.2. Проверка соответствия offline / API

14. ML-команда передаёт эталонные sales/stock CSV и результат offline predict_64 на них.
15. Backend вызывает POST с теми же файлами, сортирует оба результата по date, warehouse_name, article.
16. Проверяются одинаковые ключи и прогнозы с rtol=0, atol=1e-7 (допуск корректировать под фактическую сериализацию).
17. Повторить для нескольких cutoff, включая крайние случаи с отсутствующими stock, редкими продажами и неактивными парами.
18. Сохранить в CI smoke-тесты, а результаты полного нагрузочного прогона — в файл отчёта.

8.3. Definition of Done

- Сервис запускается в чистом Docker-окружении одной командой; модель загружается без доступа к тренировочному датасету.
- Все обязательные и согласованные дополнительные эндпоинты возвращают контрактные ответы.
- Негативные сценарии возвращают требуемые HTTP-коды и единый формат ошибок.
- POST создаёт полный грид 562 496, без пропусков/дубликатов/отрицательных значений.
- Инференс устойчив при реальном полном входе и укладывается в лимиты ТЗ по RAM/времени.
- Повторные прогоны стабильны; offline/API parity-test проходит.
- README, test_api.sh, requirements.lock или фиксированные версии и .dockerignore лежат в репозитории.

9. План задач для backend-команды

ID / приоритет	Задача	Результат приёмки
BE-01 / P0	Скелет FastAPI, конфигурация, модель на startup, /health	Успешный запуск образа, ошибка старта при отсутствии model
BE-02 / P0	CSV ingestion и типизированный валидатор	400/422 для всех негативных сценариев
BE-03 / P0	Единый Predictor, загрузка артефактов, вызов ML predict_64	Совпадение с offline на golden input
BE-04 / P0	POST /predict и текстовый CSV response	562 496 строк + 1 header
BE-05 / P0	Кэш и GET /predict с фильтрами	64 строки по паре; корректная фильтрация
BE-06 / P0	GET /predict/all CSV/JSON, streaming	Нет лишнего пика RAM на отдаче
BE-07 / P0	Dockerfile, .dockerignore, offline-run	Автономная сборка, запуск, лимиты соблюдены
BE-08 / P0	Интеграционные, негативные и parity тесты	CI/shell-тесты проходят
BE-09 / P1	Бенчмарки, логирование запросов и метаданных	Отчёт по RAM/time/model_version
BE-10 / P1	README, API-паспорта, известные ограничения	Новый разработчик повторяет запуск по README

Очередность: сначала BE-01/02/03, затем BE-04/05/06, после — Docker, тесты и документация. В ходе разработки backend ежедневно сверяет текущий контракт Predictor с ML-командой.

10. Вопросы к организаторам и проектные допущения

Открытый вопрос	Решение до ответа / последствия
Все 11×799 пары реально присутствуют в train?	Генерировать полный декартов грид по закреплённым каталогам; проверить на данных.
Допустимы ли новые article/warehouse в API?	Для строгого submission — отклонять неизвестные ID 422; динамический режим — отдельный scope.
Будущие stock известны?	Нет; использовать только историю на cutoff. Нельзя автоматически обнулять все 64 дня.

Что означает пропуск продаж?	Уточнить календарную полноту; принимать за ноль только при подтверждении.
Что означает дубликат stock?	Не суммировать; отклонять конфликтующие строки до определения правил.
Что значит «использовать все данные», если передано >180 дней?	Контракт признаков задаёт ML: rolling 180 плюс заранее согласованные long-history агрегаты при наличии. Важно сохранить offline/API parity.
Как именно scorer считает SMAPE?	В приложенном score.py нули (0,0) исключаются из средних по ряду, хотя текст ТЗ говорит включать их как 0. Уточнить у организаторов; влияет на ML, не на API.
Что делать, если GET до POST?	Предлагается 409 с понятной ошибкой; ТЗ не регламентирует.

11. Требуемый состав репозитория

```
forecast-container/
├── app/
│   ├── main.py             # маршруты /predict /predict/all /health
│   ├── predictor.py        # вызов единого ML inference
│   └── validators.py       # строгие проверки CSV и ошибок
├── src/
│   ├── data_loader.py      # парсинг / агрегация
│   ├── features.py         # общая offline+API генерация признаков
│   └── predict.py          # единый predict_64
├── model/
│   ├── lightgbm.txt        # или model.joblib
│   ├── config.json
│   └── catalogs.json
├── tests/
│   ├── test_api.sh
│   ├── test_contract.py
│   └── test_parity.py
├── Dockerfile
├── .dockerignore
├── requirements.txt
└── README.md
```

Наличие model/ обязательно: загрузка весов при старте, отсутствие доступа к train-CSV внутри контейнера. При выборе собственного формата модели название файла и команда загрузки меняются согласованно.

12. Команды для проверки

```
# Сборка и запуск

docker build -t nonton-forecast:latest .
docker run --rm --name nonton-forecast -p 8000:8000 \
  --memory=32g --cpus=8 nonton-forecast:latest

# Сервис готов
curl -f http://localhost:8000/health

# Полный прогноз из двух файлов
curl -f -X POST http://localhost:8000/predict \
```

```
  -F "sales=@data/sales_last_180.csv" \
  -F "stock=@data/stock_last_180.csv" \
  -o forecast.csv

# Весь кэшированный прогноз
curl -f 'http://localhost:8000/predict/all?format=csv' \
  -o full_forecast.csv
```

13. Критерии успеха проекта

После реализации backend-команда сдаёт: исполнимый Docker-образ, Git-репозиторий со всеми конфигурациями, валидатор CSV, четыре рабочих маршрута, потоковую выдачу полного прогноза, набор тестов и протокол замеров времени/памяти.

ГОТОВО К ПЕРЕДАЧЕ Backend отвечает за корректность и надёжность сервиса; ML — за качество предсказаний. Общая точка интеграции — versioned predict_64 и golden test: одинаковые входы должны давать одинаковые выходы offline и в Docker.

Источники и происхождение требований

3. Все положения, отмеченные «Уточнить» или «Решение», не следует выдавать за прямые правила организаторов до согласования.